

Analytical Problem Solving

Warehouse Delivery Robot Path Planning

A. PEAS Description

Component	Description
Performance Measure	Reach the destination using minimum movements while avoiding blocked cells.
Environment	5×5 warehouse grid containing free and blocked cells.
Actuators	Move the robot Up, Down, Left, or Right.
Sensors	Detect the current position, destination, blocked cells, and free cells.

Performance Measure

- Reach the package destination.
- Avoid blocked cells.
- Minimize the number of movements.
- Complete the delivery safely.

Environment

The environment is a 5×5 warehouse grid containing free cells and blocked cells.

Actuators

The robot can move Up, Down, Left, and Right.

Sensors

The robot can identify the current position, blocked cells, free cells, and goal position.

B. Goal-Based Agent

The most suitable agent for this problem is a Goal-Based Agent.

The robot has a specific goal of reaching (5,5) from the starting position (1,1). It must select a sequence of movements that leads to the destination while avoiding blocked cells.

Therefore, a goal-based agent is suitable because the robot selects actions based on achieving its specified goal.

C. Search Problem Formulation

Initial State

(1,1)

Goal State

(5,5)

Actions

{Up, Down, Left, Right}

Diagonal movement is not allowed.

Transition Model

The transition model determines the new position after an action.

Examples:

$(1,1) + \text{Right} \rightarrow (1,2)$

$(1,1) + \text{Down} \rightarrow (2,1)$

The robot cannot move into a blocked cell.

Path Cost Function

Every movement has a cost of 1.

Path Cost = Number of movements $\times$ 1

For 8 movements:

Total Cost = $8 \times 1 = 8$

D. Breadth-First Search (BFS)

BFS explores nodes level by level using a queue. Since every movement has the same cost of 1, BFS finds the shortest path.

The action order used is:

Right $\rightarrow$ Down $\rightarrow$ Left $\rightarrow$ Up

BFS Step-by-Step Expansion

Step	Expanded Node	Queue After Expansion
1	(1,1)	(1,2), (2,1)
2	(1,2)	(1,3), (2,1)
3	(2,1)	(1,3), (3,1)
4	(1,3)	(1,4)
5	(3,1)	(1,4), (3,2), (4,1)
6	(1,4)	(1,5), (3,2)
7	(3,2)	(1,5), (4,1)
8	(4,1)	(1,5), (5,1)
9	(1,5)	(5,1), (2,5)
10	(5,1)	(2,5), (5,2)
11	(2,5)	(5,2), (3,5)
12	(5,2)	(3,5), (5,3)
13	(3,5)	(5,3), (4,5)
14	(5,3)	(4,5), (5,4)
15	(4,5)	(5,4)
16	(5,4)	(5,5)
17	(5,5)	Goal reached

Final BFS Path

$(1,1) \rightarrow (1,2) \rightarrow (1,3) \rightarrow (1,4) \rightarrow (1,5) \rightarrow (2,5) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$

Total Cost

BFS Path Cost = 8

E. Depth-First Search (DFS)

DFS explores one path as deeply as possible before backtracking. It uses a stack.

The action order used is:

Right $\rightarrow$ Down $\rightarrow$ Left $\rightarrow$ Up

DFS Step-by-Step Expansion

Step	Expanded Node	Stack
1	(1,1)	(2,1), (1,2)
2	(1,2)	(2,1), (1,3)
3	(1,3)	(2,1), (1,4)
4	(1,4)	(2,1), (1,5)
5	(1,5)	(2,1), (2,5)
6	(2,5)	(2,1), (3,5)
7	(3,5)	(2,1), (4,5)
8	(4,5)	(2,1), (5,5)
9	(5,5)	Goal reached

Final DFS Path

$(1,1) \rightarrow (1,2) \rightarrow (1,3) \rightarrow (1,4) \rightarrow (1,5) \rightarrow (2,5) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$

Total Cost

DFS Path Cost = 8

Program for BFS and DFS

```
*untitled*
File Edit Format Run Options Window Help

from collections import deque

grid = [
    [0,0,0,0,0],
    [0,1,1,0,0],
    [0,0,1,0,0],
    [0,1,0,1,0],
    [0,0,0,0,0]
]

start = (0,0)
goal = (4,4)

moves = [(0,1), (1,0), (0,-1), (-1,0)]

def bfs():
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        current, path = queue.popleft()

        if current == goal:
            return path

        for dr, dc in moves:
            r = current[0] + dr
            c = current[1] + dc

            if 0 <= r < 5 and 0 <= c < 5 and grid[r][c] == 0:
                if (r,c) not in visited:
                    visited.add((r,c))
                    queue.append(((r,c), path+[(r,c)]))

def dfs():
    stack = [(start, [start])]
    visited = {start}

    while stack:
        current, path = stack.pop()

        if current == goal:
            return path

        for dr, dc in reversed(moves):
            r = current[0] + dr
            c = current[1] + dc
```

```

        for dr,dc in moves:
            r = current[0] + dr
            c = current[1] + dc

            if 0 <= r < 5 and 0 <= c < 5 and grid[r][c] == 0:
                if (r,c) not in visited:
                    visited.add((r,c))
                    queue.append(((r,c),path+[(r,c)]))

def dfs():
    stack = [(start,[start])]
    visited = {start}

    while stack:
        current,path = stack.pop()

        if current == goal:
            return path

        for dr,dc in reversed(moves):
            r = current[0] + dr
            c = current[1] + dc

            if 0 <= r < 5 and 0 <= c < 5 and grid[r][c] == 0:
                if (r,c) not in visited:
                    visited.add((r,c))
                    stack.append(((r,c),path+[(r,c)]))

bfs_path = bfs()
dfs_path = dfs()

print("BFS Path:")
print(bfs_path)
print("BFS Cost:",len(bfs_path)-1)

print("\nDFS Path:")
print(dfs_path)
print("DFS Cost:",len(dfs_path)-1)|

```

Output:

```

IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/SRINATH/Downloads/SLOT-C/AT-1.py =====
BFS Path:
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]
BFS Cost: 8

DFS Path:
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]
DFS Cost: 8
>>> |

```

F.BFS and DFS Comparison

Criteria	BFS	DFS
Completeness	Complete	Complete for finite search space with visited states
Optimality	Optimal when all step costs are equal	Not guaranteed to be optimal
Time Complexity	$O(b^d)$	$O(b^m)$
Space Complexity	$O(b^d)$	$O(bm)$
Data Structure	Queue	Stack
Search Method	Level by level	Depth first
Shortest Path	Guaranteed	Not guaranteed

Where:

- b = Branching factor
- d = Depth of the shallowest solution
- m = Maximum search depth

Suitable Algorithm

BFS is more suitable for this warehouse delivery problem because every movement has a uniform cost of 1. BFS explores the grid level by level and guarantees the shortest path.

Therefore, BFS is preferred because it can:

- Reduce travel distance.
- Reduce delivery time.
- Reduce energy consumption.
- Improve warehouse efficiency.
-

Conclusion

The warehouse delivery robot was successfully modeled as a goal-based intelligent agent. The search problem was formulated and solved using BFS and DFS. Both methods obtained a path from (1,1) to (5,5) with a total cost of 8 for the selected traversal order. Since all movements have equal cost, BFS is more suitable because it guarantees the shortest path.